

Ghid de pregătire pentru prezentarea proiectului

Chatbot Q&A pentru întrebări frecvente ale studenților · concepte, cifre și întrebări probabile

1. Proiectul în 60 de secunde (răspunsul-pilot)

Dacă profesoara spune „prezintă-mi pe scurt proiectul”, acesta este răspunsul de bază – învață-l fluent:

„Am construit un chatbot care răspunde întrebărilor frecvente ale studenților – despre secretariat, examene, burse, cazare, practică, licență. Am creat manual o bază de cunoștințe cu 40 de perechi întrebare-răspuns în 10 categorii și un set separat de 40 de reformulări pentru testare. Am implementat trei metode de căutare, de la simplu la complex: căutare prin cuvinte-cheie cu similaritate Jaccard (variante de bază), TF-IDF cu similaritate cosinus și căutare semantică prin LSA. Le-am evaluat cu recall@K și MRR: varianta de bază găsește răspunsul corect din prima în 50% din cazuri, iar cea semantică în 82.5%. Aplicația are interfață web în Streamlit, recunoaște când nu știe răspunsul și este complet reproductibilă – o comandă regenerează toată evaluarea.”

2. Cifrele pe care trebuie să le știi pe de rost

Ce	Valoare	De ce contează
Baza de cunoștințe	40 întrebări-răspunsuri, 10 categorii	creată manual; sursa datelor documentată
Set de test	40 de parafraze (una per intrare)	nevăzute la construirea indexului
recall@1	0.50 → 0.775 → 0.825	keyword → TF-IDF → LSA
recall@3	0.675 → 0.95 → 0.925	TF-IDF câștigă ușor la @3 (vezi Î. specială)
recall@5	0.80 → 0.975 → 0.975	39/40 rezolvate în top 5
MRR	0.62 → 0.858 → 0.881	media 1/rang; LSA cel mai bun
Timp de răspuns	sub 5 ms / întrebare	toate metodele; diferența e irelevantă
Componente LSA	30 (random_state=42)	hiperparametru fixat înainte de evaluare
Greșeli top-1 (LSA)	7 din 40	3 tipare de eroare – vezi secțiunea 6

3. Harta codului – ce face fiecare fișier

Fișier	Rol
data/faq.csv	baza de cunoștințe: id, categorie, întrebare, raspuns
data/intrebari_test.csv	id_asteptat + întrebare_test (40 parafraze, doar pentru evaluare)
src/utils.py	încărcarea și validarea datelor; normalize() – litere mici, fără diacritice, fără punctuație; tokenize() cu stopwords
src/search.py	cele 3 motoare: KeywordSearch (Jaccard), TfidfSearch (TF-IDF+cosinus), LsaSearch (SVD+cosinus); SbertSearch opțional; toate au metoda search(query, top_k)
src/evaluate.py	pentru fiecare întrebare de test calculează rangul răspunsului corect → recall@1/3/5, MRR; salvează metrici.csv, detalii per întrebare și 2 grafice
src/demo.py	demonstrație în linia de comandă (interactiv sau o singură întrebare)

Fișier	Rol
app.py	interfața Streamlit: chat, alegerea metodei, top-3 cu scoruri, prag de încredere

4. Concepte fundamentale (din ghidul de practică)

IA vs. ML vs. Deep Learning vs. modele generative

Inteligența Artificială = domeniul larg al sistemelor care rezolvă sarcini ce cer inteligență. **Machine Learning** = subdomeniu în care modelul învață tipare din date, în loc de reguli scrise de mână. **Deep Learning** = ML cu rețele neuronale cu multe straturi. **Modele generative** = modele care produc conținut nou (text, imagini). Proiectul meu este un sistem de **regăsire a informației** (information retrieval) cu tehnici clasice de ML/NLP – nu antrenează o rețea neuronală și nu generează text nou.

Tipuri de probleme

Clasificare – etichete discrete (spam/non-spam). **Regresie** – valoare numerică continuă (prețul unei locuințe). **Detectare de anomalii** – observații neobișnuite (fraude). **Recomandare** – elemente relevante pentru un utilizator. **Generare** – conținut nou. Problema mea seamănă cu o clasificare cu 40 de clase, dar este tratată corect ca **ranking/regăsire**: ordonez candidații după scor de similaritate, de aceea folosesc recall@K, nu accuracy.

Seturi de date și evaluarea corectă

Set de antrenament – din care modelul învață / din care se construiește indexul. **Set de validare** – pentru reglarea hiperparametrilor. **Set de test** – folosit o singură dată, la final, pentru estimarea performanței reale. Regula de aur: informația din test nu are voie să influențeze construirea sistemului (altfel apare **data leakage** și rezultate optimist-false). În proiect: indexul se construiește doar din faq.csv; parafrazele de test apar exclusiv în evaluate.py.

Overfitting, underfitting, hiperparametri

Overfitting – modelul memorează datele de antrenament și performează slab pe date noi (performanță mare pe train, mică pe test). **Underfitting** – modelul e prea simplu și performează slab peste tot. **Hiperparametri** – valori alese de om, nu învățate din date (la mine: numărul de componente LSA = 30, intervalele n-gramelor, pragul de încredere). Le-am fixat înainte de evaluare tocmai ca să nu le „potrivesc” pe setul de test – asta ar fi o formă de overfitting la nivel de evaluare.

5. Conceptele-cheie ale proiectului (cu formule și exemple)

Normalizarea textului

Toate textele trec prin: litere mici → eliminarea diacriticelor (descompunere Unicode NFKD, apoi eliminarea semnelor combinatorii: „ș”→„s”) → eliminarea punctuației → spații unice. Motiv: utilizatorii scriu frecvent fără diacritice; fără normalizare, „restanta” și „restanță” ar fi termeni diferiți. **Stopwords** (cuvinte de legătură: „de”, „la”, „cum”) se elimină doar la varianta de bază, pentru că acolo fiecare cuvânt are pondere egală; la TF-IDF nu e nevoie – IDF le penalizează automat (apar peste tot ⇒ pondere aproape zero).

Similaritatea Jaccard (varianta de bază)

$J(A, B) = |A \cap B| / |A \cup B|$ — raportul dintre cuvintele comune și totalul cuvintelor distincte. Exemplu: interogarea {program, secretariat} și întrebarea {program, secretariat, facultate} → $J = 2/3 \approx 0.67$. Este o variantă de bază bună pentru că e simplă, rapidă și complet interpretabilă – orice îmbunătățire trebuie să o bată ca să se justifice. Limita: cere potriviri exacte de cuvinte („pauză” ≠ „întrerupere” ⇒ scor 0).

TF-IDF

Fiecare text devine un vector în care ponderea unui termen = **TF** (frecvența termenului în document) × **IDF** ($\log(N/df)$ – cât de rar este termenul în colecție). Intuiție: un cuvânt e important dacă apare des în acest document, dar rar în celelalte. „bursa” diferențiază documentele; „student” apare peste tot, deci primește pondere mică. Folosesc **sublinear_tf** ($1+\log(tf)$) ca repetarea unui cuvânt să nu domine scorul. Vectorizez pe două niveluri combinate: **cuvinte** (unigrame + bigrame, ca să prind expresii precum „bursa sociala”) și **n-grame de caractere** (3-5 caractere, în interiorul cuvintelor): „restanta”, „restantele”, „restanța” împart fragmentele „res”, „est”, „sta”... ⇒ robustețe la flexiuni, diacritice și greșeli de tastare, fără stemming sau lematizare.

Similaritatea cosinus

$\cos(q, d) = (q \cdot d) / (||q|| \cdot ||d||)$ — cosinusul unghiului dintre vectori; 1 = aceeași direcție, 0 = nimic în comun. După normalizarea L2 a vectorilor, este un simplu produs scalar (așa e implementat în cod). De ce cosinus și nu distanța euclidiană? Pentru text contează **direcția** vectorului (despre ce vorbește), nu lungimea lui (câte cuvinte are). Un text lung și unul scurt despre același subiect au unghi mic între ei, dar distanță euclidiană mare.

LSA - analiza semantică latentă (TruncatedSVD)

Matricea TF-IDF (documente × termeni, rară și cu mii de dimensiuni) este proiectată prin descompunere în valori singulare trunchiată (TruncatedSVD) într-un spațiu **dens** cu 30 de dimensiuni. Fiecare dimensiune latentă grupează termeni care apar împreună în aceleași contexte – de aici „semantic”: „cămin”, „cazare”, „cameră”, „tarif” ajung apropiate, deci o interogare poate fi similară cu o întrebare din FAQ chiar **fără cuvinte comune**. Vectorii rezultați sunt **embeddings**: reprezentări numerice dense în care apropierea geometrică $\approx$ apropiere de sens. Pentru documentele indexate combin întrebarea (pondere dublă) cu răspunsul – răspunsul aduce vocabular suplimentar util. De ce 30 de componente? Avem doar 40 de documente, deci maximum teoretic e 39; 30 păstrează suficientă informație și forțează gruparea semantică; cu prea puține componente se pierde distincții, cu prea multe LSA degenerază spre TF-IDF simplu.

SBERT (extensia opțională)

sentence-transformers cu modelul multilingv paraphrase-multilingual-MiniLM-L12-v2: o rețea neuronală de tip transformer, **preantrenată pe perechi de parafraze**, care produce embeddings de propoziții. Diferența față de LSA: LSA învață doar din cele 40 de documente ale mele (co-aparițiile locale), SBERT aduce cunoaștere lingvistică din milioane de texte, deci rezolvă sinonimia totală („pauză” ~ „întrerupere”). Costul: model de ~470 MB descărcat de pe Hugging Face, inferență mai lentă. E implementat în cod (SbertSearch) și evaluabil cu --sbert, dar nu inclus în rezultatele raportate.

Metricile: recall@K și MRR

recall@K = proporția întrebărilor de test pentru care răspunsul corect apare în primele K rezultate. E metrica naturală a problemei: interfața arată primul răspuns plus două alternative, deci recall@1 și recall@3 descriu direct experiența utilizatorului. **MRR** (Mean Reciprocal Rank) = media valorilor 1/rang. Exemplu de calcul pe 3 întrebări cu rangurile 1, 2 și 4: $MRR = (1/1 + 1/2 + 1/4)/3 = 1.75/3 \approx 0.583$. MRR e mai fin decât recall: premiază pozițiile mai bune, nu doar „a intrat / n-a intrat în top K”. **De ce nu accuracy / precision / matrice de confuzie?** Nu e o clasificare cu clase fixe, ci un ranking cu 40 de candidați per interogare; accuracy ar fi exact recall@1, iar o matrice de confuzie 40×40 cu câte un exemplu pe clasă nu ar aduce informație utilă. Știu totuși ce sunt (accuracy = proporția predicțiilor corecte; precision = din ce am prezis pozitiv, cât e corect; recall clasic = din pozitivele reale, cât am găsit; F1 = media lor armonică) – se folosesc la clasificare, ex. analiza de sentimente.

Pragul de încredere

Dacă scorul celui mai bun rezultat e sub un prag (dependent de metodă, ex. 0.35 la LSA), chatbotul răspunde „nu am găsit un răspuns potrivit” și recomandă secretariatul. Ales empiric, observând scorurile tipice pentru potriviri corecte vs. întrebări din afara domeniului. E mai onest să recunoști incertitudinea decât să dai un răspuns greșit cu încredere.

6. Analiza erorilor - cele 3 tipare (cu exemplele exacte)

Tipar	Exemplu real din test	Explicație / remediu
Sinonimie totală	„Cum imi fac mailul de student?” → răspuns despre adeverințe	niciun termen comun cu „e-mail instituțional”; „student” atrage altă întrebare. Remediu: SBERT sau formulări alternative în FAQ
Categorii dense	„Trebuie neaparat sa fac practica?” → alt răspuns despre practică	întrebările din aceeași categorie sunt foarte apropiate; corectul apare aproape mereu în top 3 ⇒ justifică afișarea alternativelor
Formulări indirecte	„De cate ori pot da examen fara sa platesc?” → contestații	interogarea descrie situația fără termenul-cheie („prezentări”). Remediu: îmbogățirea FAQ cu reformulări

7. Întrebări probabile și răspunsuri pregătite

A. Despre problemă și date

Î: De ce ai ales această temă?

R: Este o problemă reală pe care o văd în jurul meu – aceleași întrebări administrative revin constant – și acoperă tot lanțul cerut de ghid: date, variantă de bază, îmbunătățiri, evaluare, interfață. În plus, regăsirea de informație e fundamentul sistemelor moderne de tip RAG.

Î: De ce ai creat datele manual în loc să folosești un dataset public?

R: Ghidul pentru Tema 3 cere explicit construirea unui fișier CSV cu întrebări și răspunsuri. Crearea manuală mi-a dat control asupra categoriilor și mi-a permis să construiesc un set de test de parafraze cu corespondență exactă (id_asteptat), fără de care recall@K nu se poate calcula.

Î: 40 de intrări nu e prea puțin?

R: Pentru demonstrarea și compararea metodelor este suficient și este o limitare recunoscută în raport. Concluziile comparative (semantic > lexical) rămân valabile la scară; pentru mii de intrări aş adăuga indexare FAISS. Colectarea întrebărilor reale ale utilizatorilor e prima direcție de extindere.

Î: Cum ai asigurat că nu evaluezi pe datele de antrenament?

R: Prin construcție: indexul se construiește exclusiv din faq.csv, iar intrebari_test.csv este citit doar în evaluate.py. Parafrazele sunt formulate diferit de întrebările din FAQ – exact scenariul real.

Î: De ce nu ai set de validare?

R: Metodele nu au parametri învățați care să necesite reglare pe validare. Singurii hiperparametri (30 de componente LSA, n-gramele, pragul) au fost fixați înainte de a rula evaluarea, tocmai ca să nu-i optimizez pe test. Cu mai multe date, aş separa un set de validare pentru reglarea lor riguroasă.

Î: De ce testele sunt scrise fără diacritice?

R: Intenționat: reflectă modul real în care tastează utilizatorii și testează robustețea normalizării. Sistemul funcționează identic cu și fără diacritice, pentru că normalizarea le elimină din ambele părți.

B. Despre metode

Î: De ce ai început cu o variantă de bază atât de simplă?

R: Ghidul o cere explicit, iar metodologic e esențială: fără o referință simplă nu pot demonstra că metodele complexe aduc valoare. Jaccard e interpretabil – pot explica exact de ce a dat un scor – și surprinzător de competitiv: 50% recall@1.

Î: Explică diferența dintre căutarea prin cuvinte-cheie și cea semantică.

R: Cea lexicală (keyword, TF-IDF) potrivește forme de cuvinte: dacă interogarea și documentul nu împart termeni, scorul e zero. Cea semantică lucrează în spațiul sensului: embeddings-urile aduc aproape texte despre același subiect chiar fără cuvinte comune. Exemplu din proiect: „Cat platesc pe luna la camin?” își găsește răspunsul despre „tariful de cazare” doar semantic.

Î: Ce face concret TruncatedSVD?

R: Descompune matricea TF-IDF în factori și păstrează doar primele 30 de direcții cu cea mai mare variație – combinații liniare de termeni care apar împreună. Proiectarea documentelor pe aceste direcții produce vectori denși (embeddings) în care termenii corelați („cazare”, „cămin”, „tarif”) contribuie la aceleași dimensiuni.

Î: De ce SVD „trunchiat” și nu PCA?

R: Sunt strâns înrudite; TruncatedSVD funcționează direct pe matrice rare (sparse) fără centrare, ceea ce e exact cazul TF-IDF. Centrarea unei matrice rare ar distruge raritatea și ar fi ineficientă.

Î: Ce este un embedding?

R: O reprezentare numerică densă a unui text într-un spațiu vectorial, construită astfel încât apropierea geometrică să corespundă apropierii de sens. La mine embeddings-urile vin din LSA (30 dimensiuni); la SBERT vin dintr-un transformer preantrenat (384 dimensiuni).

Î: De ce similaritate cosinus și nu distanță euclidiană?

R: Pentru text contează direcția vectorului (subiectul), nu lungimea (numărul de cuvinte). Cosinusul e invariant la lungime. Practic, după normalizarea L2, cosinusul e un produs scalar – rapid de calculat pentru toate documentele deodată.

Î: Cum ai trata o bază de cunoștințe cu 100.000 de intrări?

R: Calculul similarității cu toți candidații devine costisitor, deci aș folosi un index de căutare aproximativă a vecinilor (FAISS) peste embeddings – ghidul îl menționează ca pas opțional. Arhitectura mea permite asta direct, pentru că toate metodele produc vectori și scoruri.

Î: De ce nu ai folosit direct un model generativ (gen ChatGPT)?

R: Un model generativ poate halucina răspunsuri – inacceptabil pentru informații administrative oficiale. Sistemul meu returnează doar răspunsuri validate din baza de cunoștințe. Extensia corectă e RAG: regăsirea rămâne sursa de adevăr, iar modelul generativ doar reformulează răspunsul găsit – e menționată la direcții viitoare.

C. Despre evaluare și rezultate

Î: De ce recall@K și nu accuracy?

R: Problema e de ranking, nu de clasificare cu clase fixe: pentru fiecare interogare ordonez 40 de candidați. Accuracy pe top-1 ar fi identic cu recall@1, dar ar ascunde informația esențială – cât de aproape de vârf e răspunsul corect când nu e primul. recall@3 contează direct pentru interfață, care afișează 3 rezultate.

Î: Cum se calculează MRR? Dă un exemplu.

R: Media valorilor $1/\text{rang}$ pe toate întrebările de test. Exemplu: trei întrebări cu răspunsul corect pe rangurile 1, 2 și 4 $\rightarrow (1 + 0.5 + 0.25)/3 \approx 0.583$. La mine: 0.62 (keyword), 0.858 (TF-IDF), 0.881 (LSA).

Î: Interesant: TF-IDF bate LSA la recall@3 (0.95 vs 0.925). De ce?

R: Compresia LSA în 30 de dimensiuni ajută potrivirea semantică de vârf (recall@1 mai bun), dar estompează ușor distincțiile fine între întrebări foarte asemănătoare – la câteva interogări răspunsul corect alunecă de pe locul 2-3 pe 4. E un compromis clasic compresie vs. precizie fină; ambele ating 0.975 la recall@5. Faptul că pot explica acest detaliu arată că am analizat rezultatele, nu doar le-am raportat.

Î: Ce se întâmplă dacă întreb ceva complet din afara domeniului?

R: Scorul maxim rămâne sub pragul de încredere, iar chatbotul spune explicit că nu a găsit un răspuns potrivit și recomandă secretariatul – comportament pe care îl pot demonstra live.

Î: Cum garantezi reproductibilitatea?

R: requirements.txt cu versiunile bibliotecilor, random_state=42 la SVD, date incluse în repository, README cu pași de instalare și rulare, iar întreaga evaluare (tabele + grafice) se regenerează cu o singură comandă: python src/evaluate.py.

Î: Ai încercat alte valori pentru numărul de componente LSA?

R: Am fixat 30 înainte de evaluare, pe considerente de dimensiune (40 documente ⇒ max 39). O analiză de sensibilitate pe un set de validare ar fi pasul riguros următor – pot rula pe loc evaluarea cu altă valoare dacă doriți, e un parametru al clasei LsaSearch.

D. Întrebări critice / capcană

Î: Nu e cam mică diferența dintre TF-IDF și LSA ca să justifice complexitatea?

R: La recall@1 diferența e de 2 întrebări (31 vs 33 din 40) – modestă, dar tipurile de erori rezolvate sunt cele importante: potrivirile fără cuvinte comune, imposibile lexical. Pe date mai variate mă aștept ca avantajul semantic să crească; de aceea păstrez ambele în aplicație și le compar transparent.

Î: Dacă adaug o întrebare nouă în FAQ, ce trebuie refăcut?

R: Doar adăugarea rândului în faq.csv; la pornire indexul (vectorizare + SVD) se reconstruiește automat în câteva zeci de milisecunde. Nu există model antrenat persistent care să necesite reantrenare.

Î: Unde ar putea apărea overfitting în proiectul tău?

R: Dacă aș fi reglat pragul de încredere sau numărul de componente uitându-mă la rezultatele pe setul de test și alegând ce dă cifre maxime – ar fi leakage de evaluare. Am evitat asta fixând hiperparametrii înainte. Alt risc: setul de test are câte o parafrază per intrare, deci estimarea are varianță – mai multe parafraze per întrebare ar da o estimare mai stabilă.

Î: Ce ai face diferit cu încă o săptămână?

R: Aș evalua SBERT și l-aș include în comparație, aș adăuga 2-3 parafraze de test per intrare pentru estimări mai stabile, aș face o analiză de sensibilitate pentru componentele LSA și aș colecta întrebări de la colegi pentru un test cu adevărat extern.

Î: Poți explica o linie anume din cod?

R: Da – recomand să parcurgi search.py și evaluate.py linie cu linie înainte de prezentare. Punctele fierbinți: FeatureUnion (combină cei doi vectorizatori), l2_normalize + produs scalar (= cosinus), np.argsort(-scores) (ordonare descrescătoare), clip(upper=10) în grafic (rangurile mari se afișează ca 10).

8. Glosar rapid (răspunsuri într-o propoziție)

Corpus — colecția de documente în care se caută (cele 40 de întrebări FAQ).

Token — unitatea de bază a textului după împărțire (de regulă un cuvânt).

Stopwords — cuvinte frecvente fără conținut informativ („de”, „la”, „cum”).

n-gramă — secvență de n elemente consecutive (cuvinte sau caractere).

Vector rar (sparse) — vector cu majoritatea valorilor zero (TF-IDF brut).

Vector dens — vector cu valori pe toate dimensiunile (embeddings LSA/SBERT).

Normalizare L2 — împărțirea vectorului la lungimea sa ⇒ lungime 1; cosinus devine produs scalar.

SVD — descompunerea unei matrice în factori ortogonali ordonați după importanță.

Data leakage — informație din test care se strecoară în construirea sistemului ⇒ rezultate false.

Pipeline — lanț de pași de preprocesare + model, aplicați consecvent (concept scikit-learn).

RAG — Retrieval-Augmented Generation: regăsire de documente + reformulare generativă.

FAISS — bibliotecă de căutare rapidă (aproximativă) a celor mai apropiați vectori.

random_state — sămânța generatorului aleator $\Rightarrow$ rezultate identice la fiecare rulare.

Streamlit cache_resource — construiește motoarele o singură dată, nu la fiecare mesaj.

9. Checklist înainte de prezentare

- ☐ Rulează local tot: `pip install -r requirements.txt` $\rightarrow$ `python src/evaluate.py` $\rightarrow$ `streamlit run app.py`.
- ☐ Pregătește 3 demonstrații live: (1) o întrebare parafrazată care merge doar semantic, (2) comparația keyword vs. LSA pe aceeași întrebare, (3) o întrebare din afara domeniului (pragul de încredere).
- ☐ Modifică ceva mic înainte (adaugă o întrebare în FAQ, schimbă `n_components`) și observă efectul – cel mai rapid mod de a stăpâni codul.
- ☐ Recitește raportul – întrebările profesoarei pleacă de obicei din ce ai scris acolo.
- ☐ Știi pe de rost: 40/10/40, recall@1 0.50 $\rightarrow$ 0.775 $\rightarrow$ 0.825, MRR 0.881, 30 componente, 3 tipare de eroare.
- ☐ Dacă nu știi un răspuns, spune ce știi sigur și cum ai verifica – nu inventa.